



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G5

System Integration and Architecture

System Design Document (SDD)

Project Title: CheckMate Academy

Prepared By: Paradela, Lawrence Anthony D.

Version: 1

Date: 2/20/2026

Status: Final

REVISION HISTORY TABLE

Versio n	Date	Author	Changes Made	Status
0.1	2/20/2026	Lawrence Anthony D. Paradela	Initial draft	Draft
0.2	2/27/2026	Lawrence Anthony D. Paradela	Added API specifications	Review
0.3	[Date]	[Your Name]	Updated database design	Review
0.4	[Date]	[Your Name]	Added UI/UX designs	Review
0.5	[Date]	[Your Name]	Incorporated feedback	Revised
0.6	[Date]	[Your Name]	Final review and corrections	Final
1	[Date]	[Your Name]	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY	4
1.0 INTRODUCTION	5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION	5
3.0 NON-FUNCTIONAL REQUIREMENTS	8
4.0 SYSTEM ARCHITECTURE	9
5.0 API CONTRACT & COMMUNICATION	10
Authentication Endpoints	11
User Registration	11
User Login	11
6.0 DATABASE DESIGN	13
7.0 UI/UX DESIGN	14
8.0 PLAN	17

EXECUTIVE SUMMARY

1.1 Project Overview

CheckMate Academy is a specialized platform designed to bridge the gap between chess students and professional coaches. Built using **React** for the web and **Supabase** for the backend, it enables real-time lesson scheduling, performance tracking, and secure communication.

1.2 Objectives

1. Provide a seamless **Real-time Scheduling** system for chess lessons.
2. Implement a cloud-native architecture using React (frontend) and Supabase (backend).
3. Utilize PostgreSQL Row-Level Security (RLS) for strict data privacy between users.
4. Design a responsive user interface that works consistently across desktop and mobile browsers.

1.3 Scope

Included Features:

- User registration and authentication (role-based: Coach vs. Student).
- Coach directory with filtering by ELO rating and availability.
- Real-time lesson booking system.
- Dashboard for tracking upcoming lessons and past match reviews.
- Responsive web interface.
- PostgreSQL database with relational schema.

Excluded Features:

- Real payment gateway integration.
- In-app video conferencing (will rely on external links like Google Meet).
- Native Android/iOS applications (focusing on responsive web MVP).
- Advanced analytics dashboard.

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the CheckMate Academy system. It provides detailed requirements, architectural decisions, API contracts, database design, and an implementation roadmap to guide development and ensure seamless integration between the React frontend and Supabase backend.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: CheckMate Academy

Domain: Education / Scheduling Platform

Primary Users:

1. Students (Chess learners)
2. Coaches (Experienced players/instructors)

Problem Statement: Students struggle to find and schedule dedicated time with qualified chess coaches without heavy administrative back-and-forth.

Solution: A centralized platform focusing on real-time availability, direct booking, and clear tracking of a student's chess progress.

2.2 Core User Journeys

Journey 1: Student Books a Lesson

1. The user visits the web application and creates a "Student" account.
2. Browse the directory of available coaches.
3. Views a coach's profile and available time slots.
4. Selects a time slot and confirms the booking.
5. Receives booking confirmation on their dashboard.

Journey 2: Coach Sets Availability

1. Coach logs in with existing credentials.
2. Navigates to their scheduling dashboard.
3. Opens time slots for the upcoming week.
4. Views new booking requests from students.
5. Approves or adjusts upcoming lesson statuses.

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication (Register, login, logout via Supabase Auth).
2. Coach directory (List, detail, search).
3. Real-time booking system (Select slot, confirm, update).
4. User dashboards (View upcoming lessons).

SHOULD HAVE

1. ELO rating tracking on user profiles.
2. Basic input validation feedback.
3. Responsive design for all screen sizes.

COULD HAVE

1. Lesson notes or tactical puzzle assignments.
2. Real-time notifications for new bookings.

WON'T HAVE

1. Real payment processing.
2. Multi-language support.

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login.
- **Fields:** Email, Password, Role (Student/Coach), Lichess/Chess.com Username.
- **Security:** Managed entirely by Supabase Auth (JWT tokens, secure password hashing).
- **API Endpoints:** POST /auth/register, POST /auth/login, POST /auth/logout

Feature: Coach Directory

- **Screens:** Directory Listing, Coach Profile.
- **Display:** Grid layout, avatars, ELO rating, hourly rate, bio.
- **Functions:** Search by name, filter by ELO rating, view availability.
- **API Endpoints:** GET /coaches, GET /coaches/{id}, GET /coaches/search

Feature: Availability Management

- **Screens:** Coach Schedule Settings.
- **Functions:** Add available time slots, remove time slots, set recurring weekly hours.
- **Persistence:** Database storage in the availability table linked to the coach.
- **API Endpoints:** GET /coaches/{id}/availability, POST /availability, DELETE /availability/{id}

Feature: Lesson Booking System

- **Screens:** Booking Calendar, Booking Confirmation Modal.
- **Functions:** Select open time slot, confirm booking, cancel pending request.
- **Process:** Validate slot availability, create lesson record, lock time slot.
- **API Endpoints:** POST /lessons, GET /lessons/{id}, DELETE /lessons/{id}

Feature: User Dashboard

- **Screens:** Student Dashboard, Coach Dashboard.
- **Functions:** View upcoming lessons, view past lessons, update lesson status (accept/complete).
- **Access Control:** Role-based data retrieval (Supabase Row-Level Security ensures users only see their own lessons).
- **API Endpoints:** GET /lessons/student/{id}, GET /lessons/coach/{id}, PATCH /lessons/{id}/status

2.5 Acceptance Criteria

AC-1: Successful User Registration

- **Given** I am a new user
- **When** I enter a valid email, strong password, and select the "Student" role
- **And** confirm the password matches
- **And** click "Create Account"
- **Then** my account should be created in Supabase Auth
- **And** a corresponding profile should be created in the **profiles** table
- **And** I should be redirected to the Student Dashboard

AC-2: Coach Availability Management

- **Given** I am logged in as a Coach
- **When** I navigate to my schedule settings

- **And** I select a specific day (e.g., Monday) and time range (e.g., 2:00 PM - 5:00 PM)
- **And** I click "Save Availability"
- **Then** the time slots should be saved to the database
- **And** my public profile should immediately display these available slots to students

AC-3: Lesson Booking Flow

- **Given** I am logged in as a Student
- **When** I view a Coach's profile
- **And** I select an available time slot
- **And** I click "Confirm Booking"
- **Then** a new record should be created in the `lessons` table with a "pending" status
- **And** the selected time slot should no longer be available for other students to book
- **And** the pending lesson should appear on my Student Dashboard

AC-4: Coach Lesson Approval

- **Given** I am logged in as a Coach
- **When** I view my Coach Dashboard
- **And** I see a new lesson request with a "pending" status
- **And** I click "Accept"
- **Then** the lesson status should update to "confirmed" in the database
- **And** the updated status should be reflected on both the Coach and Student dashboards

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API/Supabase response time: ≤ 2 seconds for 95% of requests.
- Web page load time: ≤ 3 seconds on broadband connections.
- Real-time synchronization: Booking status updates via Supabase Realtime must reflect on connected clients within 1 second.
- Concurrency: Support up to 100 concurrent users (students and coaches) interacting with the platform simultaneously.
- Database queries complete within 500ms for scheduling conflicts and profile lookups.

3.2 Security Requirements

- HTTPS for all communications between the React frontend and Supabase backend.

- Authentication managed entirely by Supabase Auth using secure JWT tokens.
- Data access is strictly controlled using PostgreSQL Row-Level Security (RLS) policies (e.g., users can only view their own private lesson details; coaches can only edit their own availability).
- SQL injection prevention handled inherently by PostgREST and Supabase client parameterization.
- XSS (Cross-Site Scripting) protection handled natively by React's DOM rendering.
- API rate limiting enforced by Supabase default project settings to prevent abuse.

3.3 Compatibility Requirements

- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions).
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+).
- **Operating Systems:** Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+, iOS 15+, Android 12+.

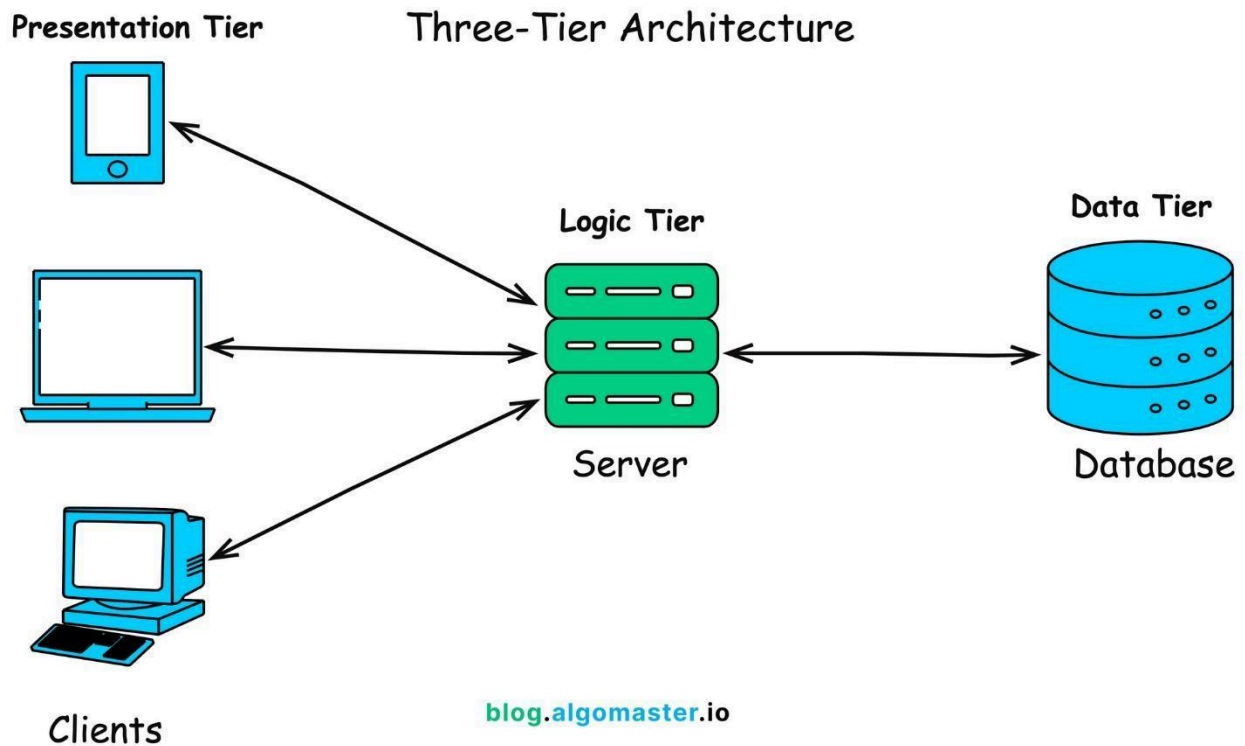
3.4 Usability Requirements

- New students must be able to complete their first lesson booking within 5 minutes of landing on the site.
- WCAG 2.1 Level AA compliance for web accessibility (proper contrast ratios, ARIA labels for calendar grids).
- Consistent navigation across all pages (Student Dashboard, Coach Directory).
- Clear error messages with recovery options (e.g., "This time slot is no longer available. Please select another time.").
- Touch targets a minimum 44x44px on mobile devices, especially for the calendar scheduling interface.
- Full keyboard navigation support for the booking calendar and accessibility tools.

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin, XML-Based, Retrofit, Room
- **Build Tools:** Maven (Backend), npm (Web), Gradle (Android)
- **Deployment:** Render (Backend), Render (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[project_ref].supabase.co/rest/v1 (for database) and /auth/v1 (for authentication)
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header, alongside the Supabase apikey header.
Response Structure	<pre>{ "success": boolean, "data": object null, "error": { "code": string, "message": string, "details": object null }, "timestamp": string }</pre>

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre>{ "email": "<email>", "password": "<password>", "full_name": "<full_name>", "role": "<student_or_coach>", "chess_username": "<lichess_or_chesscom_username>", "current_elo": <integer> }</pre>
Response Structure	<pre>{ "success": boolean, "data": { "user": { "id": "<uuid>", "email": "<email>", "full_name": "<full_name>", </pre>

	<pre> "role": "<student_or_coach>", "chess_username": "<username>", "current_elo": <integer> }, "accessToken": "<token>", "refreshToken": "<token>" }, "error": { "code": string, "message": string, "details": object null }, "timestamp": string } </pre>
--	---

User Login

Description	Authenticate a Student or Coach
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre> { "email": "<email>", "password": "<password>" } </pre> Response Structure <pre> { "success": boolean, "data": { "user": { "id": "<uuid>", "email": "<email>", "full_name": "<full_name>", "role": "<student_or_coach>", "current_elo": <integer> }, "accessToken": "<token>", "refreshToken": "<token>" }, "error": { "code": string, "message": string, "details": object null }, "timestamp": string } </pre>

	}
Response Structure	<pre>{ "success": boolean, "data": { "user": { "email": <email>, "firstname": <firstname>, "lastname": <lastname>, "role": <role>, }, "accessToken": <token>, "refreshToken": <token>, }, "error": { "code": string, "message": string, "details": object null }, "timestamp": string }</pre>

Fetch Coach Availability

Description	Retrieves all open time slots for a specific coach.
API URL	/availability?coach_id=eq.{id}
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token (JWT)
Request Payload	
Response Structure	<pre>{ "success": boolean, "data": [{ "id": <avail_id>, "coach_id": <coach_id>, "day_of_week": <integer>, "start_time": <time>, "end_time": <time> }], "error": { "code": string, "message": string, "details": object null } }</pre>

	<pre> }, "timestamp": string } </pre>
--	---------------------------------------

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

Example 1	<pre> json { "success": false, "data": null, "error": { "code": "AUTH-001", "message": "Invalid credentials", "details": "Email or password is incorrect" }, "timestamp": "2024-01-28T10:30:00Z" } </pre>
Example 2	<pre> { "success": false, "data": null, "error": { "code": "VALID-001", "message": "Validation failed", "details": { "email": "Email is required", "password": "Must be at least 8 characters" } }, } </pre>

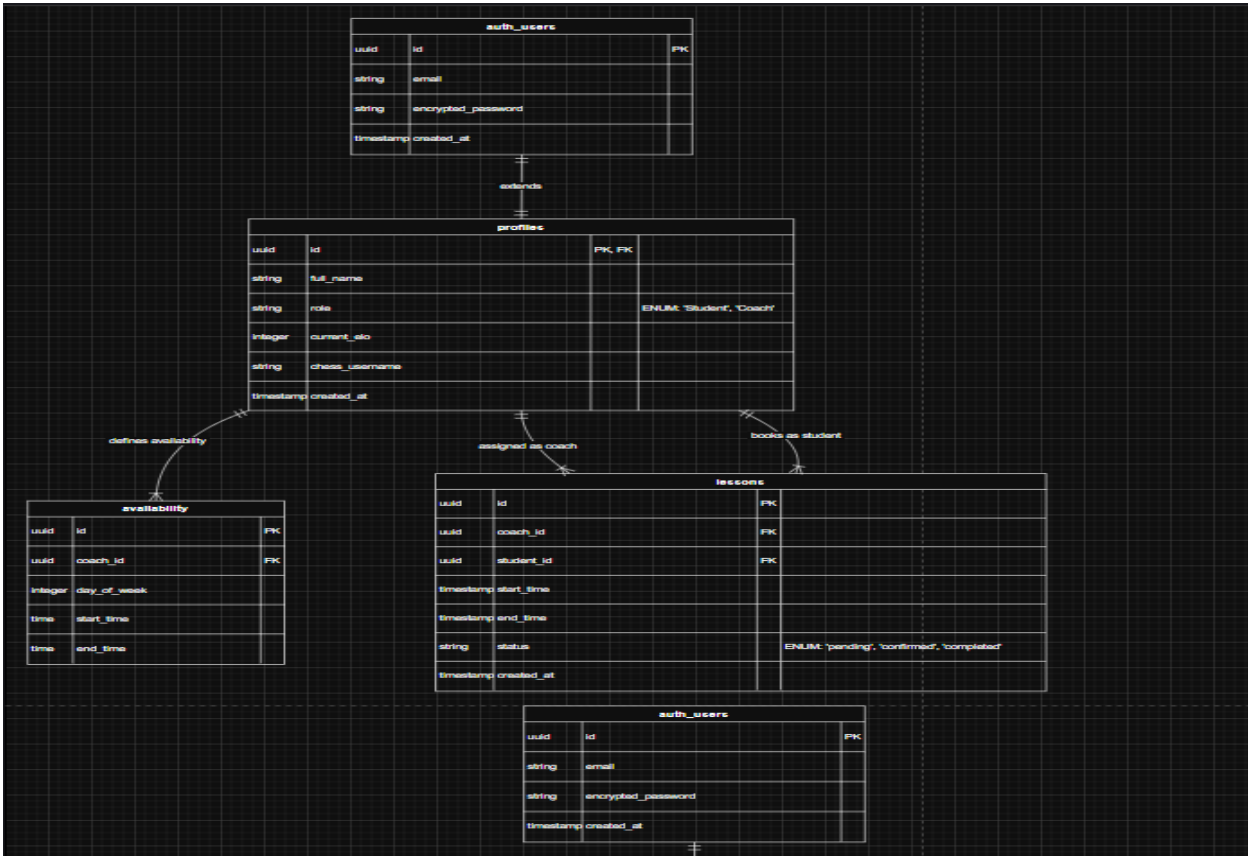
	<pre>"timestamp": "2024-01-28T10:30:00Z" }</pre>
--	--

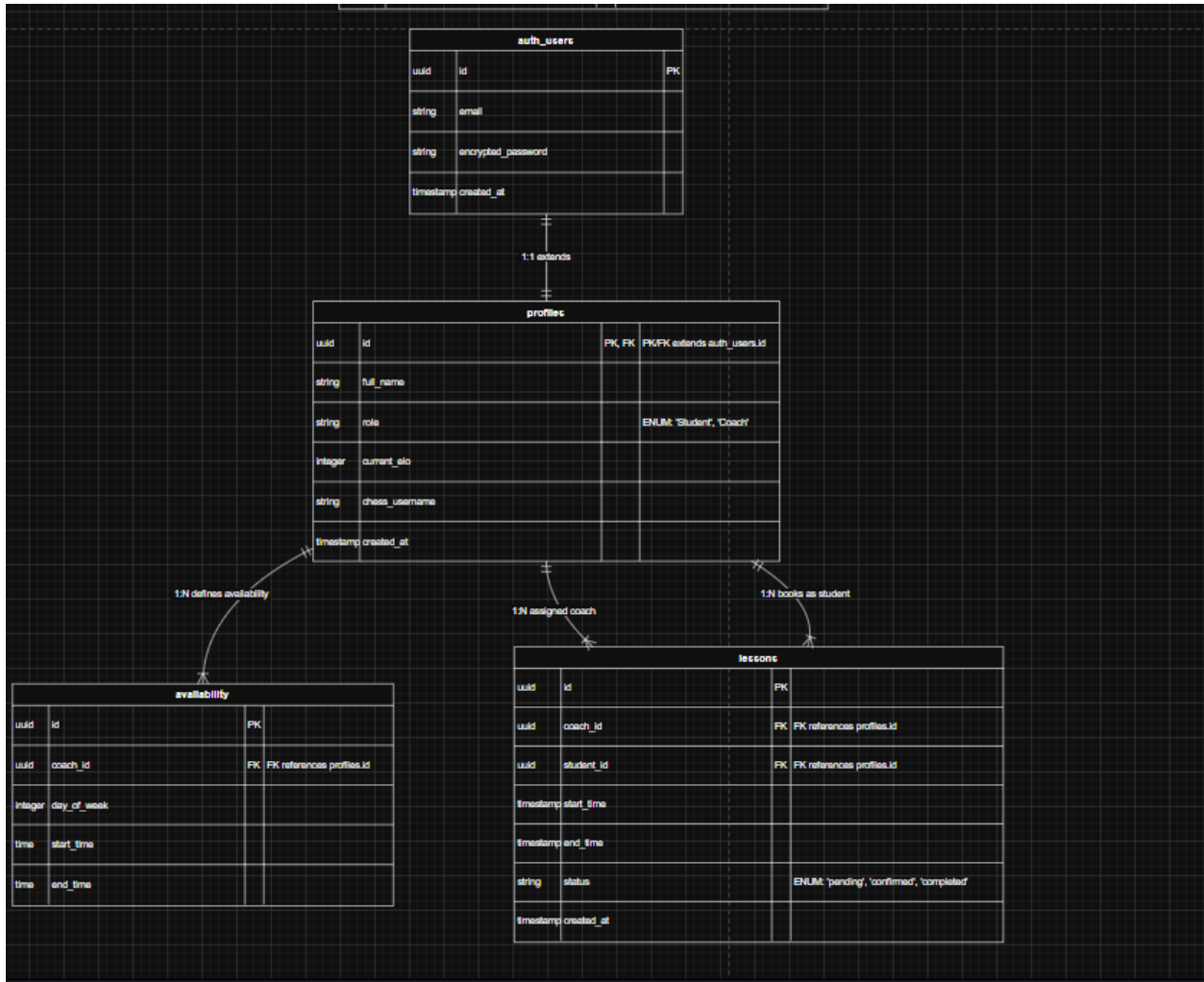
Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUSINESS-001: Time slot unavailable
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram





Detailed Relationships:

- **One-to-One:** auth.users ↔ profiles (Each auth user has exactly one public profile)
- **One-to-Many:** profiles (Coach) → availability (Coach can have multiple availability slots)
- **One-to-Many:** profiles (Coach) → lessons (Coach can have multiple lessons)
- **One-to-Many:** profiles (Student) → lessons (Student can have multiple lessons)

Key Tables:

1. **auth.users** - User accounts and authentication managed by Supabase
2. **profiles** - Public user profiles (Coach/Student details and chess credentials)
3. **availability** - Coach schedule and open time slots
4. **lessons** - Lesson bookings and current statuses

Table Structure Summary:

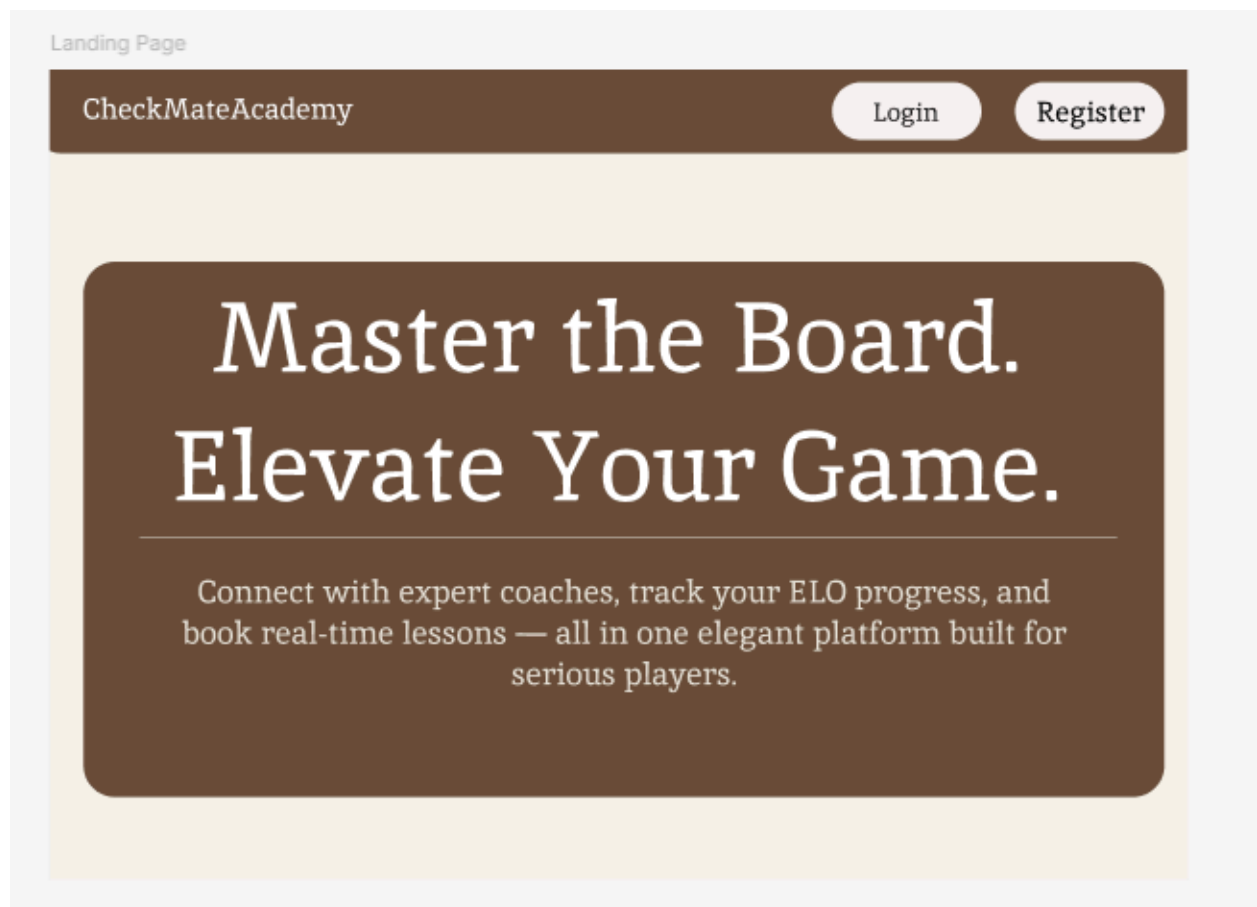
- **auth.users:** id, email, encrypted_password, created_at
- **profiles:** id, full_name, role, current_elo, chess_username, created_at
- **availability:** id, coach_id, day_of_week, start_time, end_time
- **lessons:** id, coach_id, student_id, start_time, end_time, status, created_at

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

<https://www.figma.com/design/AyBxINygMn7T5o9Vewb1KE/Untitled?node-id=0-1&t=XqGYmH0Oow5ak4Wk-1a>

Landing Page



Login

Login Page

CheckMateAcademy

Login Register

Welcome Future Grand Master

Username

Password

Login

Register Page

Register Page

CheckMateAcademy

Login Register

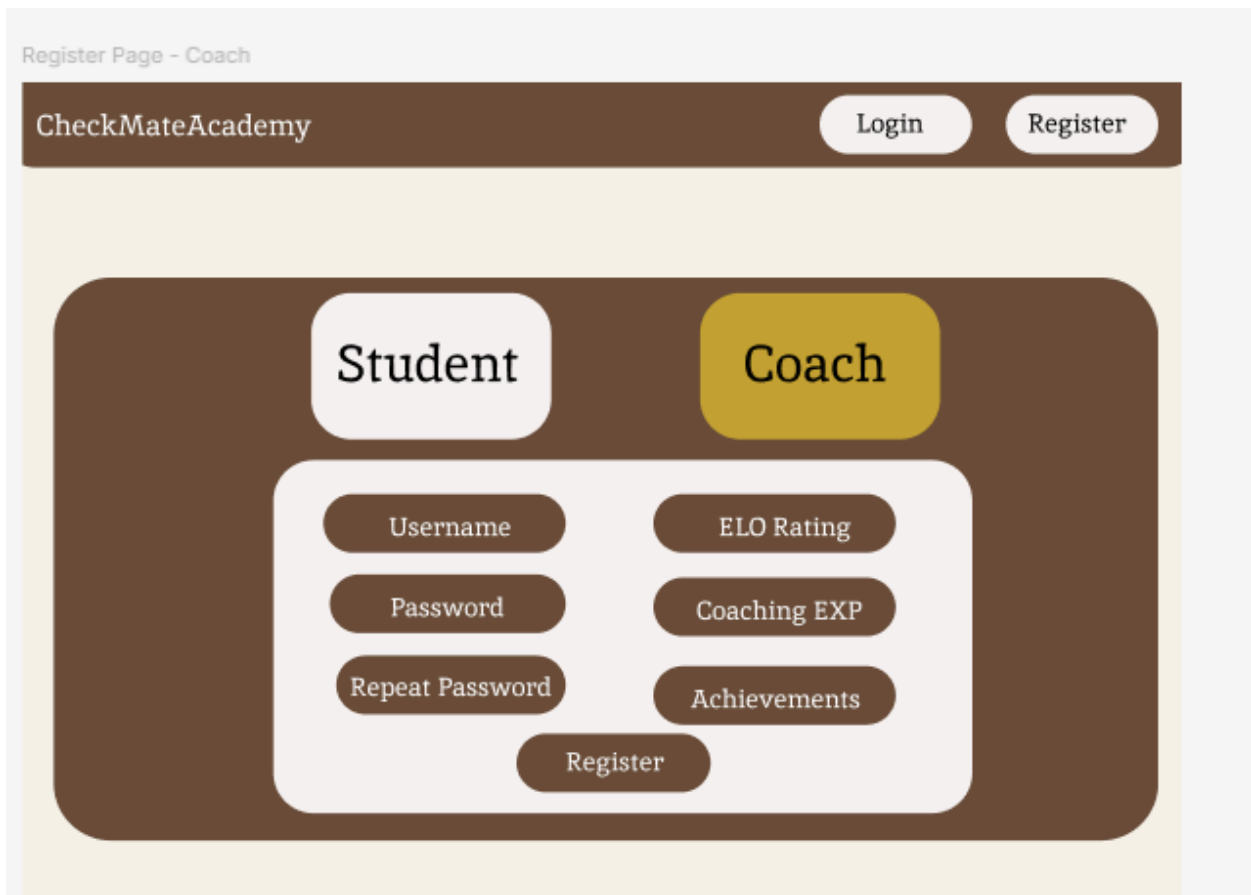
Student Coach

Username

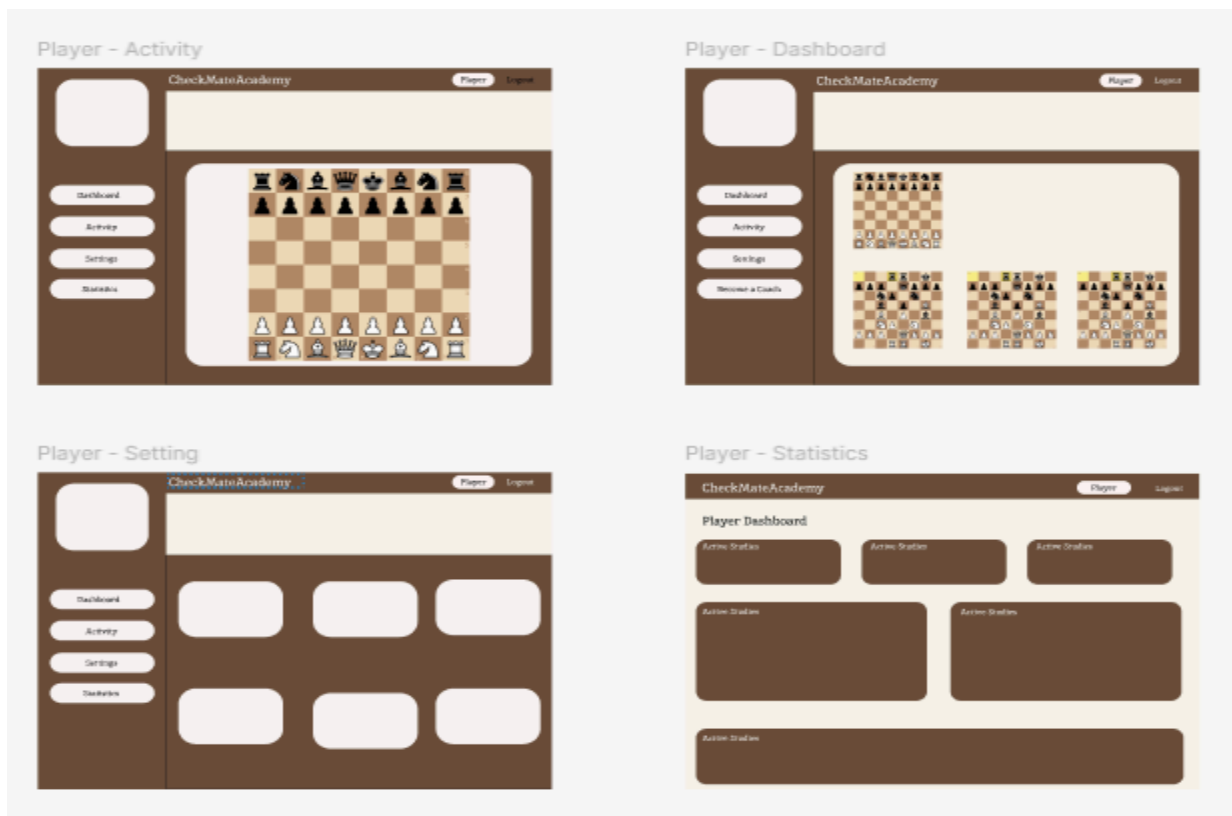
Password

Repeat Password

Register



Player Pages



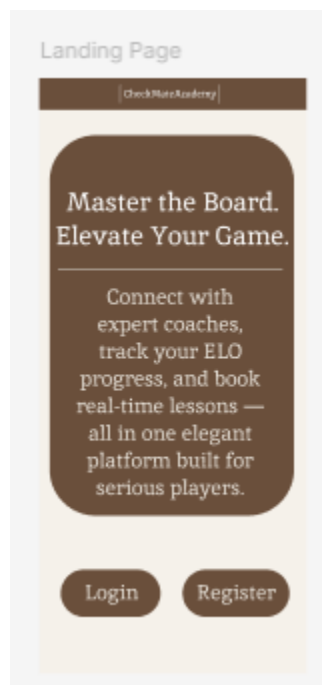
Coach Pages



7.2 Mobile Application Wireframes

<https://www.figma.com/design/AyBxINYgMn7T5o9Vewb1KE/Untitled?node-id=0-1&t=XqGYmH0Oow5ak4Wk-1>

Landing Page



Login Page

Login Page

CheckMateAcademy

LOGIN

Username

Password

Login

Register Page

Register

CheckMateAcademy

Register

Student Coach

Username

Password

Repeat Password

Register

Register - Coach

CheckMateAcademy

Register

Student Coach

Username

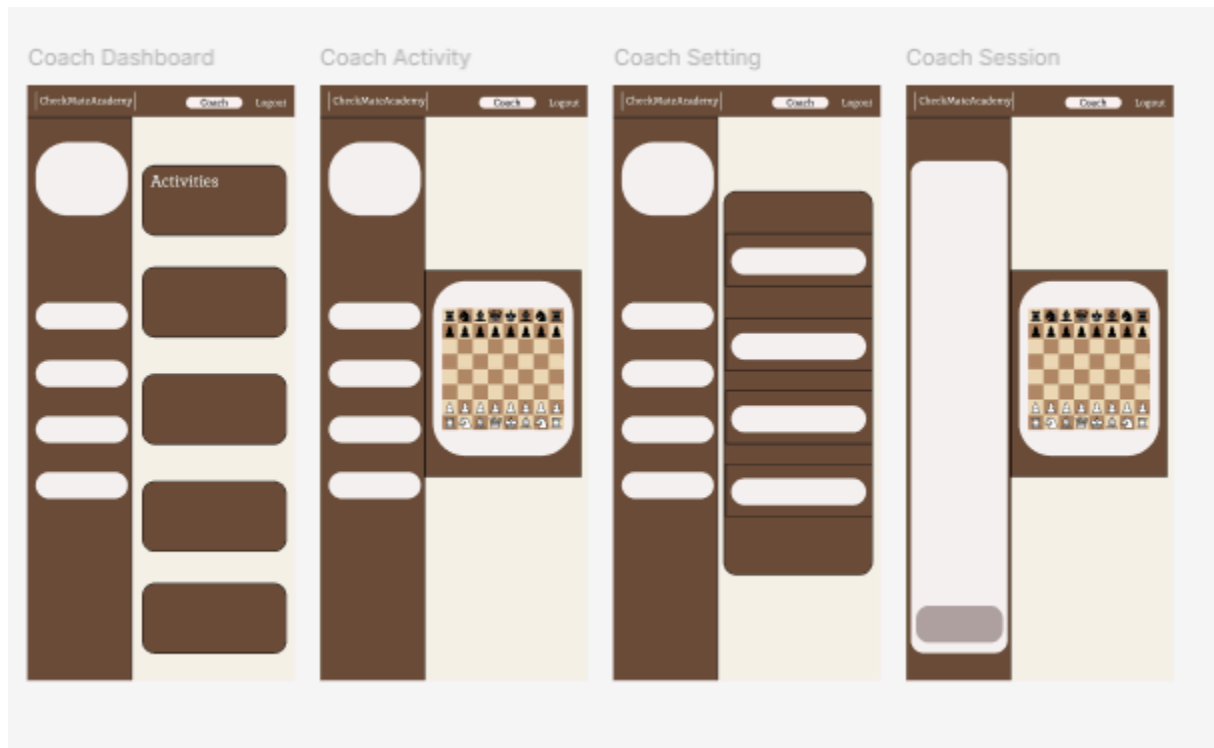
Password

Repeat Password

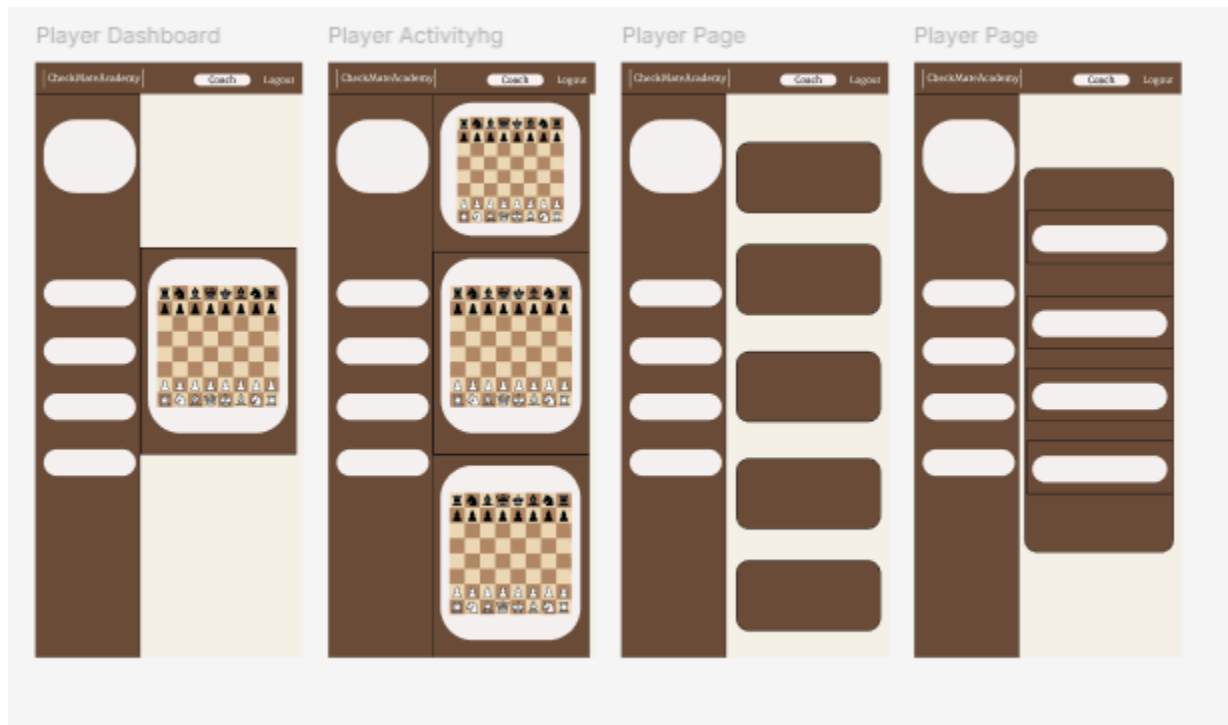
ELO Rating

Register

Coach Pages



Player Pages



Mobile-Specific Features:

- **Touch Optimization:** Touch-optimized buttons and selectable time slots (minimum 44x44px for accessibility).
- **Gesture Support:** Swipe to navigate between lesson days; pull-to-refresh on the dashboard for new booking requests.
- **Caching:** Offline caching for coach avatars and basic layout elements to speed up load times.
- **Responsive Navigation:** The desktop sidebar collapses into a bottom navigation bar or hamburger menu for mobile users to prioritize screen real estate for the chess board.
- **Input Simplification:** Simplified date/time picker forms for mobile lesson booking.

Design System:

- **Colors:** Primary (#6B4F3A), Secondary (#F5F1EA), Success (#10B981), Error (#EF4444)
- **Typography:** Inika font family, responsive sizing
- **Spacing:** 8px grid system
- **Components:** Consistent buttons, inputs, cards, modals
- **Responsive:** Mobile-first approach, breakpoints at 640px, 768px, 1024px

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation

Day 3-4: Complete FRS and NFR

Day 5-7: System architecture design

Week 2: Detailed Design

Day 1-2: API specification

Day 3-4: Database design

Day 5-6: UI/UX wireframes

Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Supabase project setup and configuration

Day 2: Database schema and entity creation (profiles, availability)

Day 3: Supabase Auth implementation and user roles

Day 4: Row-Level Security (RLS) policies setup

Day 5: Coach directory endpoints and functions

Week 4: Core Features

Day 1: Coach availability management

Day 2: Lesson booking system

Day 3: Search and filtering (by Elo rating/name)

Day 4: Error handling and booking conflict validation

Day 5: API documentation and testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React setup with Tailwind CSS

Day 2: Authentication pages (login, register)

Day 3: Coach directory listing page

Day 4: Coach profile and schedule view

Day 5: Lesson booking calendar implementation

Week 6: Complete Web Features

Day 1: Student and Coach dashboards

Day 2: Lesson status management (Accept/Complete)

Day 3: Real-time updates integration

Day 4: Responsive design polish

Day 5: Supabase API integration and testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup and project structure

Day 2: Authentication screens

Day 3: Coach browsing and search

Day 4: Availability calendar view

Day 5: Supabase SDK integration

Week 8: Complete Mobile App

Day 1: Lesson booking flow

Day 2: User dashboards and lesson management

Day 3: UI polish and animations

Day 4: Testing on emulator/device

Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing across platforms

Day 2: Bug fixes and optimization

Day 3: Security review

Day 4: Performance testing

Day 5: Documentation updates

Week 10: Deployment

Day 1: Backend finalization (Supabase Cloud)

Day 2: Web app deployment (Vercel)

Day 3: Mobile APK distribution

Day 4: Final testing

Day 5: Project submission

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)
2. Coach availability API (Week 3-4)
3. Lesson booking functionality (Week 4)
4. Dashboard and status management (Week 6)
5. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements

